

Băm và tập hợp (Hashing and Sets)

PHỤ THUỘC A/05 cho xác suất và nghịch lý ngày sinh; B/06 cho mẫu "tổng tiền tố cộng bảng băm". Chuẩn bị cho B/12 (băm chuỗi) và C/20 (ngẫu nhiên hoá).

TÓM TẮT

Băm là cách đổi một khoá bất kỳ thành một chỉ số mảng, để tra cứu trở thành truy cập trực tiếp. Nó cho $O(1)$ trung bình cho thêm, xoá và tìm — rẻ hơn hẳn cây tìm kiếm — nhưng đánh đổi ba thứ: không có thứ tự, không có đảm bảo trường hợp xấu nhất, và dễ bị tấn công có chủ đích. Chương này đi qua cách chọn hàm băm, hai cách xử lý va chạm, ý nghĩa của hệ số tải, nghịch lý ngày sinh và hệ quả của nó với băm chuỗi, cùng hai cấu trúc xác suất là Bloom filter và bộ đếm số phần tử phân biệt. Nếu chỉ nhớ một điều: bảng băm nhanh vì nó *giả định dữ liệu không thù địch*, và trong lập trình thi đấu cũng như trong hệ thống công khai, giả định đó bị phá bằng cách trộn một hạt giống ngẫu nhiên vào hàm băm.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm băm (hash function)** — hàm biến một khoá thành một số nguyên trong khoảng $[0, m)$. Yêu cầu duy nhất về mặt đúng đắn: khoá bằng nhau phải cho cùng giá trị; yêu cầu về hiệu năng: các khoá khác nhau nên rải đều.
- **va chạm (collision)** — hai khoá khác nhau cho cùng giá trị băm. Không tránh được vì số khoá khả dĩ luôn lớn hơn số ô.
- **hệ số tải (load factor)** — tỉ số giữa số phần tử và số ô của bảng, $\alpha = n/m$. Quyết định chi phí trung bình.
- **ngịch lý ngày sinh** — trong k giá trị lấy ngẫu nhiên từ N khả năng, xác suất có hai giá trị trùng nhau trở nên đáng kể khi $k \approx \sqrt{N}$.
- **băm đa thức cho chuỗi** — coi chuỗi như đa thức theo cơ số b , lấy giá trị theo modulo m : $H(s) = \sum s_i b^i \bmod m$. Cho phép tính băm của mọi đoạn con trong $O(1)$ sau tiền xử lý.
- **Bloom filter** — cấu trúc trả lời "phần tử này có trong tập không" với bộ nhớ rất nhỏ, không bao giờ báo sai "không có", nhưng đôi khi báo nhầm "có".

CÂU HỎI MỞ ĐẦU

- Q1. Bảng băm cho $O(1)$ trung bình còn cây cân bằng cho $O(\log n)$. Vì sao không phải lúc nào cũng dùng bảng băm?
- Q2. Vì sao một lời giải dùng `unordered_map` có thể bị đánh trượt trên Codeforces trong khi cùng lời giải đó dùng `map` thì qua?
- Q3. Băm chuỗi bằng một modulo $10^9 + 7$ nghe rất an toàn vì xác suất trùng chỉ 10^{-9} . Vì sao trong bài có 10^5 chuỗi thì lập luận đó sai?
- Q4. Bloom filter có thể báo nhầm "có" nhưng không bao giờ báo nhầm "không có". Cấu trúc nào của nó tạo ra sự bất đối xứng đó, và vì sao bất đối xứng lại hữu ích?
- Q5. Trong hệ phân tán, thêm một máy chủ vào cụm làm hầu hết khoá phải chuyển chỗ nếu dùng `hash mod n`. Cách sửa là gì?

Bài toán tra cứu là bài toán phổ biến nhất trong lập trình: cho một khoá, tìm giá trị đi kèm. Nếu khoá là số nguyên nhỏ, lời giải hiển nhiên là dùng mảng — khoá chính là chỉ số, tra cứu tốn một phép truy cập. Toàn bộ ý tưởng băm là mở rộng lời giải hiển nhiên ấy cho khoá bất kỳ: nếu khoá không phải chỉ số, hãy ép nó thành chỉ số bằng một hàm.

Cái giá của việc ép đó là va chạm, vì số khoá khả dĩ luôn nhiều hơn số ô. Toàn bộ kỹ thuật băm — chọn hàm, xử lý va chạm, chọn kích thước bảng — chỉ là cách sống chung với va chạm sao cho chi phí trung bình vẫn gần $O(1)$. Và có một tình tiết mà giáo trình hay bỏ qua nhưng lại quyết định trong thực tế: mọi phân tích “trung bình” ở đây đều giả định dữ liệu không được chọn bởi người muốn hại bạn.

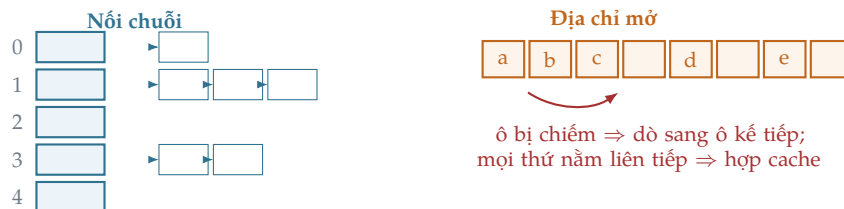
1.1 Hàm băm và hai cách sống chung với va chạm

Một hàm băm tốt cần hai tính chất: tính được nhanh, và rải các khoá thực tế đều trên các ô. Tính chất thứ hai quan trọng hơn về ngoài của nó, vì các khoá thực tế thường có cấu trúc — toạ độ trên lưới, chuỗi có tiền tố chung, số bội của một hằng — và một hàm băm ngây thơ sẽ dồn chúng vào ít ô.

Khi va chạm xảy ra, có hai cách xử lý.

Nối chuỗi: mỗi ô giữ một danh sách các phần tử băm vào đó. Đơn giản, chịu được hệ số tải lớn hơn 1, nhưng mỗi phần tử tốn thêm một con trỏ và việc lần theo danh sách không thân thiện với cache.

Địa chỉ mở: mọi phần tử nằm ngay trong mảng; nếu ô đã bị chiếm thì dò sang ô kế tiếp theo một quy tắc. Bộ nhớ gọn, truy cập tuần tự nên rất hợp cache — các bảng băm nhanh nhất trong thực tế đều dùng cách này. Cái giá: hệ số tải phải giữ dưới khoảng 0,7 nếu không chi phí bùng nổ, và việc xoá phần tử phức tạp hơn (phải đánh dấu “đã xoá” thay vì để trống).



Hình 1.1. Hai cách xử lý va chạm. Nối chuỗi đơn giản và chịu được tải cao, nhưng mỗi lần lần theo con trỏ là một lần có nguy cơ trượt cache. Địa chỉ mở giữ mọi thứ trong một mảng liên tiếp nên nhanh hơn đáng kể trên máy thật, đổi lại phải giữ hệ số tải thấp và xử lý xoá bằng cách đánh dấu.

Về chi phí: với nối chuỗi, độ dài danh sách trung bình là α , nên chi phí tìm là $O(1 + \alpha)$. Với địa chỉ mở và dò tuyến tính, chi phí trung bình tăng vọt khi α tiến gần 1 — đó là lý do bảng băm phải *tự mở rộng*: khi vượt ngưỡng tải, cấp phát bảng lớn gấp đôi và băm lại toàn bộ. Chi phí băm lại là $O(n)$ nhưng khấu hao thành $O(1)$ mỗi thao tác, đúng lập luận nhân đôi ở chương A/03.

1.2 Chỗ mà giáo trình nói “trung bình” và thực tế nói “kẻ tấn công”

Đây là mục quan trọng nhất chương, và cũng là mục hay bị bỏ.

Mọi phân tích trên đều giả định các khoá rải đều trên các ô. Nhưng hàm băm là một hàm *tất định*: nếu ai đó biết hàm băm của bạn, họ có thể chuẩn bị một tập khoá mà tất cả cùng băm vào

một ô. Khi đó bảng băm thoái hoá thành một danh sách liên kết, và mọi thao tác thành $O(n)$. Với $n = 10^5$ phần tử, lời giải $O(n)$ trên mỗi thao tác trở thành 10^{10} — quá thời gian chắc chắn.

Điều này không phải chuyện lý thuyết. Trong lập trình thi đấu, các bộ test chống `unordered_map` của C++ được viết ra chính bằng cách này và rất phổ biến. Trong hệ thống thật, hiện tượng tương tự từng được dùng để tấn công máy chủ web: gửi một yêu cầu chứa hàng nghìn tham số cùng băm vào một ô làm máy chủ tiêu tốn thời gian gấp hàng nghìn lần bình thường.

Cách chữa giống nhau ở cả hai bối cảnh: *làm cho hàm băm không đoán trước được*, bằng cách trộn vào một hạt giống ngẫu nhiên chọn lúc chạy. Về mặt lý thuyết, ý tưởng này có tên là *băm phổ dụng* — chọn ngẫu nhiên một hàm từ một họ hàm có tính chất “với hai khoá bất kỳ, xác suất va chạm nhỏ” — và điểm mấu chốt là ngẫu nhiên nằm ở *lựa chọn của bạn*, không ở dữ liệu. Nhờ vậy đảm bảo vẫn đúng ngay cả khi dữ liệu do đối thủ chọn, miễn là họ không biết hạt giống. Đây là một ví dụ mẫu mực cho toàn bộ chương C/20: ngẫu nhiên hoá không phải để hy vọng gặp may, mà để *tước bỏ khả năng dự đoán của đối thủ*.

CẠM BẮY

Trong C++, `unordered_map` với khoá số nguyên dùng một hàm băm rất đơn giản và có thể đoán được, nên nó bị đánh trượt thường xuyên trên các kỳ thi. Cách chữa tiêu chuẩn là cung cấp hàm băm riêng có trộn hạt giống lấy từ đồng hồ hệ thống. Trong ngôn ngữ khác, hãy kiểm tra xem thư viện chuẩn có ngẫu nhiên hoá hạt giống theo tiến trình hay không — nhiều ngôn ngữ hiện đại đã làm sẵn việc này sau các đợt tấn công vào máy chủ web.

1.3 Bảo đảm mạnh hơn: băm cuckoo và băm bảng tra

Băm phổ dụng cứu được trường hợp xấu *kỳ vọng*, nhưng vẫn không hứa gì về một thao tác cụ thể: một lần tra vẫn có thể phải lần qua một chuỗi dài. Với hệ thống có hạn kỳ thời gian — bộ định tuyến, bộ điều khiển, phần cứng — kỳ vọng là không đủ. Hai kết quả sau cho thấy có thể đòi hỏi nhiều hơn, và cả hai đều đáng biết vì chúng đổi cách nghĩ chứ không chỉ thêm một cấu trúc.

Băm cuckoo[1] lật ngược câu hỏi: thay vì cho phép một khoá nằm ở nhiều chỗ tùy theo lịch sử va chạm, hãy quy định mỗi khoá chỉ được nằm ở *đúng hai* vị trí, cho bởi hai hàm băm. Khi đó tra cứu là hai lần đọc bộ nhớ, không hơn — $O(1)$ trong *trường hợp xấu nhất*, và xoá cũng đơn giản vì không có ô “đã xoá” nào phải đánh dấu. Cái giá đôn hết sang phép chèn: nếu cả hai vị trí đều bị chiếm, ta *đá* kẻ đang ngồi ở một trong hai ô sang vị trí thay thế của nó, và kẻ bị đá lại đá tiếp người khác. Chuỗi đá này kết thúc nhanh nếu bảng còn đủ rộng — điều kiện quen thuộc là hệ số tải dưới $1/2$ — còn nếu nó dài quá ngưỡng thì ta bốc lại hai hàm băm mới và xây lại bảng. Kết quả: chèn tốn $O(1)$ kỳ vọng theo nghĩa khấu hao, đổi lấy bộ nhớ rộng gấp đôi và một cái đuôi hiếm hoi rất đắt.

Điểm đáng học nằm ở chỗ đánh đổi được *dời* chứ không biến mất: cùng một bài toán, nhưng chi phí bị đẩy hết từ phía đọc sang phía ghi. Đó là lựa chọn đúng cho các bảng viết một lần đọc rất nhiều lần, và là lựa chọn sai cho tài ghi liên tục.

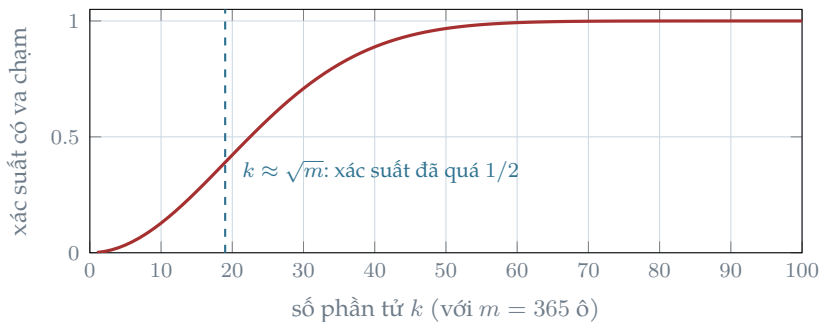
Băm bảng tra[2] trả lời một câu hỏi khác: hàm băm phải “ngẫu nhiên” tới mức nào thì các phân tích lý thuyết mới còn đúng? Cách làm đơn giản tới mức đáng ngờ — cất khoá thành vài ký tự, mỗi ký tự tra một bảng số ngẫu nhiên đã dựng sẵn, rồi XOR tất cả lại. Hàm như vậy chỉ đọc lập bậc ba, thấp hơn nhiều so với mức mà các chứng minh cổ điển đòi hỏi, nên theo lý thuyết cũ thì nó không đủ tốt. Pătraşcu và Thorup chứng minh rằng nó vẫn cho các chặn tập trung kiểu Chernoff, và nhờ đó dò tuyến tính lẫn băm cuckoo chạy đúng như phân tích với một hàm băm chỉ tốn vài phép tra bảng và một phép XOR.

Ba dòng đáng mang theo từ mục này. Một, “ngẫu nhiên hoá” không phải một mức duy nhất mà là cả một thang, và câu hỏi kỹ thuật luôn là *cần độc lập tới bậc mấy*. Hai, hàm băm rẻ kèm một phân tích tinh vi thường thắng hàm băm đắt kèm một phân tích thô. Ba, khi bài toán đòi bảo đảm cho *từng* thao tác chứ không cho trung bình, hãy hỏi xem có cấu trúc nào chuyển chi phí sang phía còn lại không — đây là mẫu tư duy dùng lại được, không riêng gì bảng băm.

1.4 Nghịch lý ngày sinh và băm chuỗi

Băm chuỗi là công cụ mạnh: coi chuỗi như một đa thức, ta tính được giá trị băm của *mọi* đoạn con trong $O(1)$ sau khi tiền xử lý $O(n)$, nên so sánh hai đoạn con bất kỳ trở thành $O(1)$. Rất nhiều bài chuỗi khó sụp xuống thành dễ nhờ điều đó (B/12).

Nhưng có một cái bẫy xác suất mà nhiều người tính sai. Nếu modulo là $10^9 + 7$, xác suất hai chuỗi cụ thể trùng băm là khoảng 10^{-9} — nghe rất an toàn. Sai lầm là ở chỗ bài toán thường so sánh *rất nhiều* cặp. Với k chuỗi, số cặp là $k^2/2$, nên xác suất có ít nhất một va chạm xấp xỉ $k^2/(2m)$. Với $k = 10^5$ và $m = 10^9$, con số đó là khoảng 5, tức là gần như chắc chắn có va chạm. Đây chính là nghịch lý ngày sinh: va chạm trở nên đáng kể khi số phần tử đạt cỡ $\sqrt{m}$, chứ không phải cỡ m .



Hình 1.2. Nghịch lý ngày sinh với $m = 365$. Trục giác nói phải cần cỡ $m/2$ phần tử mới có va chạm; thực tế chỉ cần cỡ $\sqrt{m}$, tức khoảng 23 người. Hệ quả trực tiếp cho băm chuỗi: modulo 10^9 chỉ “an toàn” tới cỡ $3 \cdot 10^4$ chuỗi, nên bài có 10^5 chuỗi cần hai modulo hoặc một modulo cỡ 10^{18} .

Hai cách chữa. Dùng *hai modulo* và coi cặp giá trị là mã băm — xác suất va chạm trở thành tích của hai xác suất, đủ nhỏ cho mọi bài thực tế. Hoặc dùng *một modulo* cỡ 10^{18} với phép nhân cẩn thận để không tràn. Và như mục trước: cơ sở của đa thức phải chọn *ngẫu nhiên lúc chạy*, vì băm chuỗi với cơ sở cố định cũng bị dựng test phá.

1.5 Cấu trúc xác suất: chấp nhận sai một chiều để tiết kiệm bộ nhớ

Khi tập dữ liệu quá lớn để lưu, có một hướng đi rất khác: chấp nhận trả lời sai với xác suất nhỏ, đổi lấy bộ nhớ nhỏ hơn nhiều bậc.

Bloom filter là ví dụ kinh điển. Ta giữ một mảng bit và k hàm băm. Thêm phần tử: bật k bit tương ứng. Hỏi phần tử: nếu có bit nào bằng 0 thì chắc chắn phần tử chưa từng được thêm; nếu mọi bit đều bằng 1 thì *có lẽ* đã thêm — có thể các bit đó do những phần tử khác bật lên. Sự bất đối xứng ấy chính là điều làm nó hữu ích: nó dùng ở vị trí bộ lọc đứng trước một phép tra cứu đắt tiền, nơi “chắc chắn không có” cho phép bỏ qua ngay, còn “có lẽ có” chỉ tốn thêm một lần kiểm tra thật. Cơ sở dữ liệu dùng nó để tránh đọc đĩa vô ích; hệ thống mạng dùng nó để lọc yêu cầu.

Đếm số phần tử phân biệt là bài toán họ hàng. Đếm chính xác cần bộ nhớ tỉ lệ với số phần tử phân biệt; nhưng nếu chấp nhận sai số vài phần trăm thì có thể dùng vài kilobyte cho hàng tỉ phần tử.

Ý tưởng gốc rất đẹp: nếu các giá trị băm rải đều, thì giá trị băm nhỏ nhất mà bạn nhìn thấy nói cho bạn biết bạn đã nhìn bao nhiêu phần tử phân biệt — càng nhiều phần tử thì cực tiểu càng nhỏ. Cấu trúc HyperLogLog[3] là bản tinh chỉnh của ý tưởng đó và được dùng rộng rãi trong các hệ thống đo lường.

1.6 Phản biện, nhược điểm và rủi ro triển khai

Bảng băm thường được ca tụng là cấu trúc thần thánh cho phép truy cập $O(1)$. Nhưng con số $O(1)$ đó đi kèm một hợp đồng ngầm có rất nhiều điều khoản bất lợi về phần cứng, bảo mật và tính ổn định.

Phản biện: ảo tưởng về $O(1)$ của thư viện chuẩn. Trong C++, `std::unordered_map` là một trong những container gây thất vọng nhất về hiệu năng thực tế. Thư viện chuẩn bắt buộc dùng phương pháp tách chuỗi (chaining) để đảm bảo iterator không bị vô hiệu hoá khi băm lại. Hệ quả là mỗi phần tử được bọc trong một nút danh sách liên kết cấp phát động riêng biệt trên heap. Trên các kích thước dữ liệu vừa và nhỏ ($N \leq 100$), chi phí cấp phát bộ nhớ và hiện tượng trượt cache liên tục (pointer chasing) khiến `std::unordered_map` chạy chậm hơn đáng kể so với `std::map` (cây đỏ-đen), dù về mặt tiệm cận cây đỏ-đen tốn $O(\log N)$. Chỉ các bảng băm dùng địa chỉ mở hiện đại với mảng phẳng (như `absl::flat_hash_map` của Google hay `robin_hood::unordered_map`) mới thực sự đạt được tốc độ của phần cứng hiện đại.

Nhược điểm cốt lõi về tài nguyên và kiến trúc:

- *Không có thứ tự và tính địa phương:* Bảng băm phá huỷ hoàn toàn thứ tự của các khoá. Nó bất lực trước các truy vấn cơ bản như “tìm khoá nhỏ nhất lớn hơn x ”, “duyệt các phần tử theo thứ tự tăng dần”, hay “truy vấn đoạn $[l, r]$ ”. Bất kỳ bài toán nào có yêu cầu về khoảng cách hay lân cận đều buộc phải dùng cây nhị phân tìm kiếm ($B/09$) hoặc cây chỉ số ($B/10$).
- *Chi phí ẩn của khoá phức tạp:* Thời gian tính băm tỉ lệ thuận với độ dài của khoá. Băm một chuỗi dài L ký tự tốn chi phí $O(L)$. Do đó, $O(1)$ của bảng băm thực chất là $O(L)$; coi thao tác này là một phép tính hằng số trên chuỗi dài là sai lầm làm chương trình chậm hơn dự tính hàng chục lần.
- *Lựa chọn sai lầm khi miền giá trị nhỏ:* Khi tập khoá là các số nguyên nằm trong một khoảng cố định (ví dụ $0 \dots 10^6$), dùng bảng băm là một quyết định kỹ thuật tồi. Một mảng tĩnh `int a[1000005]` chỉ tốn 4 MB, không bao giờ có va chạm, không tốn chi phí tính toán hàm băm và tận dụng tuyệt đối bộ nhớ đệm CPU.

Rủi ro vận hành và các lỗi sập hệ thống nghiêm trọng:

- *Tấn công suy biến (Hash Collision DoS / Test phá):* Trong các thư viện chuẩn (như `libstdc++` của GNU), hàm băm mặc định cho số nguyên chỉ là hàm đồng nhất (`hash(x) = x`), và kích thước bảng thường là các số nguyên tố cố định có thể đoán trước. Đối thủ hoặc người ra đề chỉ cần đưa vào các số nguyên cách nhau đúng một chu kỳ bảng, toàn bộ dữ liệu sẽ rơi vào cùng một bucket duy nhất. Bảng băm thoái hoá thành danh sách liên kết, chi phí mỗi thao tác từ $O(1)$ nhảy vọt lên $O(n)$, đẩy tổng thời gian lên $O(n^2)$ và làm chương trình bị quá thời gian (TLE) hoặc sập máy chủ vì nghẽn CPU (HashDoS). Cách phòng thủ bắt buộc: dùng hàm băm tùy biến kết hợp với hạt giống ngẫu nhiên lấy từ đồng hồ hệ thống (`chrono::steady_clock`) lúc chạy chương trình.
- *Gai độ trễ khi băm lại (Rehashing Latency Spike):* Khi số phần tử vượt ngưỡng hệ số tải (load factor, mặc định 1.0), bảng băm buộc phải cấp phát mảng mới lớn gấp đôi và phân bổ lại toàn bộ phần tử cũ. Bước băm lại này tốn chi phí $O(n)$. Trong các hệ thống điều khiển thời gian thực (như robot, xe tự hành, giao dịch tài chính), một thao tác đột ngột tốn vài chục mili-giây sẽ làm vỡ hạn kỳ điều khiển chu kỳ kín (deadline miss) và gây tai nạn nghiêm trọng.

- *Hiện tượng rò rỉ mộ phần (Tombstone Accumulation)*: Trong các bảng băm dùng địa chỉ mở (open addressing), khi xoá một phần tử ta không thể xoá sạch ô nhớ vì sẽ làm đứt chuỗi dò tìm của các phần tử chen sau; ta buộc phải đánh dấu ô đó là “đã xoá” (tombstone). Nếu hệ thống thực hiện hàng triệu thao tác chen và xoá xen kẽ, các tombstone sẽ lấp kín bảng, làm chuỗi dò tìm dài ra vô tận và hiệu năng tra cứu tụt dốc thê thảm dù số phần tử thực tế trong bảng rất ít.

CẠM BẦY

Không bao giờ dùng `std::unordered_map<long long, int>` mặc định trong các kỳ thi có hack (như Codeforces) hoặc trên các API công khai tiếp nhận đầu vào của người dùng mà không có custom hash. Việc tạo dữ liệu đối kháng để ép bảng băm chạy $O(n^2)$ là cực kỳ dễ dàng và được tự động hoá hoàn toàn bởi đối thủ.

1.7 Gốc rễ: từ bảng ký hiệu của trình dịch hợp ngữ tới hệ phân tán

Băm ra đời cùng lúc với những trình biên dịch đầu tiên, và vì một lý do rất cụ thể. Trình dịch hợp ngữ cần một *bảng ký hiệu*: ánh xạ tên biến sang địa chỉ. Tên biến là chuỗi, bộ nhớ thì đánh số, nên cần một câu nối — và câu nối đó là hàm băm. Ý tưởng được ghi nhận trong một ghi chú nội bộ của Hans Peter Luhn tại IBM năm 1953, và kỹ thuật địa chỉ mở xuất hiện ngay sau đó trong các trình dịch hợp ngữ giữa thập niên 1950. Đây là một ví dụ điển hình cho mẫu lặp đi lặp lại trong lịch sử ngành: cấu trúc dữ liệu quan trọng thường sinh ra từ nhu cầu của công cụ hệ thống, không từ lý thuyết.

Phần lý thuyết đến sau và giải một vấn đề thật. Suốt hai thập niên, người ta biết băm nhanh “trung bình” nhưng không có cách nào đảm bảo điều đó khi dữ liệu không ngẫu nhiên. Năm 1979, Carter và Wegman đề xuất băm phổ dụng: thay vì hy vọng dữ liệu ngẫu nhiên, hãy *chọn hàm băm một cách ngẫu nhiên*. Đây là một chuyển dịch tư duy quan trọng và nó lặp lại ở nhiều nơi khác trong quyển sách này — quicksort với chốt ngẫu nhiên (C/14) là cùng một ý.

Bloom filter (1970) sinh ra từ ràng buộc bộ nhớ của thời đó: kiểm tra một từ có nằm trong từ điển gạch nổi hay không, khi từ điển không nạp hết vào bộ nhớ được. Bài toán cũ đó biến mất, nhưng cấu trúc thì sống rất khoẻ trong các hệ thống hiện đại, ở đúng vai trò “lọc trước một phép tra cứu đắt tiền”.

Cuối cùng, một gốc rễ hiện đại hơn và có ảnh hưởng lớn: *băm nhất quán*, do nhóm của Karger đề xuất năm 1997 cho bài toán bộ nhớ đệm web phân tán. Vấn đề rất thực tế: nếu ánh xạ khoá tới máy chủ bằng hash mod n thì thêm hay bớt một máy làm gần như *toàn bộ* khoá đổi chỗ. Băm nhất quán xếp cả khoá lẫn máy chủ lên một vòng tròn và giao mỗi khoá cho máy chủ kế tiếp theo chiều kim đồng hồ; khi thêm một máy, chỉ các khoá thuộc một cung bị ảnh hưởng. Kỹ thuật này về sau trở thành nền của nhiều hệ cơ sở dữ liệu phân tán, và nó là câu trả lời cho câu hỏi mở đầu thứ năm.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1953 — Luhn (IBM)	Bảng ký hiệu của trình dịch: ánh xạ tên biến (chuỗi) sang địa chỉ (số).	Ý tưởng băm; ra đời từ nhu cầu công cụ hệ thống chứ không từ lý thuyết.
Giữa thập niên 1950 — các trình dịch hợp ngữ	Bộ nhớ rất hẹp, không đủ chỗ cho danh sách liên kết mỗi ô.	Địa chỉ mở với dò tuyến tính; cách cài vẫn nhanh nhất trên phần cứng hiện đại vì hợp cache.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
1970 — Bloom	Kiểm tra từ có trong từ điển không, khi từ điển không nạp hết vào bộ nhớ.	Bloom filter: chấp nhận sai một chiều để giảm bộ nhớ nhiều bậc; nay dùng làm bộ lọc trước truy cập đĩa.
1979 — Carter & Wegman	“Trung bình nhanh” không còn đúng khi dữ liệu do người khác chọn.	Băm phổ dụng: ngẫu nhiên hoá <i>lựa chọn hàm</i> , không phụ thuộc dữ liệu; nền cho cách chống test phá ngày nay.
1997 — Karger và cộng sự	Bộ nhớ đệm web phân tán: thêm một máy chủ làm mọi khoá đổi chỗ.	Băm nhất quán; trở thành cơ sở phân mảnh dữ liệu của nhiều hệ phân tán hiện đại.
2007 — Flajolet và cộng sự	Đếm số phần tử phân biệt trong luồng hàng tỉ bản ghi với vài kilobyte.	HyperLogLog; hoàn thiện dòng nghiên cứu ước lượng lực lượng bắt đầu từ thập niên 1980.

1.8 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Hệ phân tán	Băm nhất quán, phân mảnh dữ liệu	Cùng hàm băm, khác mục tiêu: ở đây không phải để tra cứu nhanh mà để <i>ổn định ánh xạ</i> khi số máy thay đổi. Bài học: khi tập ô có thể thay đổi, $\text{mod } n$ là lựa chọn tệ.
Hệ thống kiểm soát phiên bản	Git định danh mọi đối tượng bằng mã băm nội dung	Băm dùng làm <i>địa chỉ</i> chứ không làm chỉ số: nội dung giống nhau thì tự động chung một tên. Điều kiện để dùng được là hàm băm mật mã, tức va chạm phải khó tạo ra có chủ đích — yêu cầu mạnh hơn hẳn băm thông thường.
Mật mã	Hàm băm mật mã, cây Merkle, chữ ký	Cùng tên gọi nhưng yêu cầu khác hẳn: ở đây bắt buộc <i>không ai tạo được va chạm</i> , kể cả khi cố tình và có nhiều tài nguyên. Vì vậy chúng chậm hơn nhiều và không nên dùng làm hàm băm cho bảng băm.
Cơ sở dữ liệu	Nối bằng băm (hash join), chỉ mục băm, Bloom filter chặn đọc đĩa	Cùng đánh đổi với chương này: chỉ mục băm nhanh hơn chỉ mục cây cho truy vấn bằng, nhưng vô dụng cho truy vấn khoảng — đó là lý do B-tree vẫn là mặc định (B/09).
Nhận dạng ảnh, tìm gần đúng	Băm nhảy cảm cục bộ (LSH): khoá gần nhau thì băm giống nhau	Đảo ngược yêu cầu của chương này. Băm thường muốn rải đều để tránh va chạm; LSH <i>cố ý</i> tạo va chạm cho các phần tử tương tự, để tìm hàng xóm gần trong không gian nhiều chiều.
SLAM và nhận dạng địa điểm	Túi từ thị giác, cây từ vựng, mô tả nhị phân	Cùng ý tưởng LSH: gán mỗi đặc trưng ảnh vào một “ô” để so khớp nhanh giữa hàng nghìn khung hình. Điểm giống quan trọng: cả hai đều chấp nhận sai sót và bù lại bằng bước kiểm chứng hình học phía sau — kiến trúc “lọc rẻ trước, kiểm đất sau” giống hệt Bloom filter.
Chống trùng lặp dữ liệu	Lưu trữ theo nội dung, phát hiện tệp trùng	Băm để nhận diện thay vì để tra cứu. Cùng lưu ý về nghịch lý ngày sinh: số đối tượng càng lớn thì độ dài mã băm cần thiết càng phải tăng theo bình phương.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì $O(1)$ của bảng băm là trung bình chứ không phải đảm bảo, và vì nó mất thứ tự. Ba tình huống buộc phải dùng cây: bài cần truy vấn “khoá gần nhất”, “lớn hơn hoặc bằng x ” hay duyệt theo thứ tự; hệ thống có hạn kỳ thời gian thực không chịu được một thao tác đột ngột $O(n)$; và trường hợp khoá do đối thủ chọn mà bạn không ngẫu nhiên hoá được hàm băm. Ngoài ra, khi khoá là số nguyên trong khoảng nhỏ thì mảng thường thắng cả hai.
- A2. Vì hàm băm mặc định cho số nguyên trong nhiều cài đặt là hàm đồng nhất hoặc gần như vậy, và kích thước bảng có thể đoán được. Người viết test chọn một tập khoá cùng số dư theo kích thước bảng, làm mọi khoá dồn vào một ô; bảng băm thoái hoá thành danh sách và mỗi thao tác thành $O(n)$. Cây cân bằng không có điểm yếu này vì $O(\log n)$ của nó là đảm bảo cho mọi dữ liệu. Cách chữa là trộn hạt giống ngẫu nhiên lấy lúc chạy vào hàm băm.
- A3. Vì cái đáng lo không phải một cặp mà là *mọi cặp*. Với k chuỗi có $k^2/2$ cặp, nên xác suất có ít nhất một va chạm xấp xỉ $k^2/(2m)$. Với $k = 10^5$ và $m = 10^9$ thì con số này lớn hơn 1, tức gần như chắc chắn có va chạm. Đây là nghịch lý ngày sinh: ngưỡng nguy hiểm là $\sqrt{m}$, không phải m . Chữa bằng hai modulo hoặc một modulo cỡ 10^{18} .
- A4. Bất đối xứng đến từ chỗ Bloom filter chỉ *bật* bit chứ không bao giờ tắt: nếu một bit cần thiết đang bằng 0 thì chắc chắn phần tử chưa từng được thêm, còn mọi bit bằng 1 thì có thể do các phần tử khác bật. Điều đó hữu ích vì nó đặt cấu trúc vào đúng vai trò bộ lọc: câu trả lời “chắc chắn không” cho phép bỏ qua ngay một phép tra cứu đắt tiền, còn câu trả lời “có lẽ có” chỉ tốn thêm một lần kiểm tra thật — sai lầm không gây kết quả sai, chỉ gây thêm việc.
- A5. Băm nhất quán. Thay vì hash mod n , xếp cả khoá lẫn máy chủ lên một vòng tròn giá trị băm, và giao mỗi khoá cho máy chủ đầu tiên gặp theo chiều kim đồng hồ. Thêm một máy chỉ làm các khoá trong một cung phải chuyển, tức trung bình $1/n$ số khoá thay vì gần như toàn bộ. Trong thực tế người ta còn đặt nhiều “nút ảo” cho mỗi máy để tải rải đều hơn.

NÓI LẠI THẬT ĐƠN GIẢN

Băm chỉ là cách biến một cái tên thành một số ngắn tử. Nếu tử của bạn có 100 ngắn và bạn có quy tắc “lấy tên, tính ra một số từ 0 tới 99”, thì cất và tìm đồ đều chỉ mất một bước: tính số rồi mở đúng ngắn. Nhanh hơn hẳn việc mở lần lượt từng ngắn.

Rắc rối duy nhất là hai cái tên khác nhau có thể ra cùng một số ngắn. Có hai cách sống chung. Cách thứ nhất: mỗi ngắn treo một cái túi, ai vào cùng ngắn thì bỏ chung túi, lúc tìm thì lục túi. Cách thứ hai: nếu ngắn đã có người, đi sang ngắn kế bên cho tới khi gặp ngắn trống. Cách thứ hai nhanh hơn trong thực tế vì bạn đi bộ dọc dãy tử, không phải lục túi.

Có hai điều dễ nhầm và đáng nhớ. Thứ nhất, đụng ngắn xảy ra sớm hơn ta tưởng rất nhiều: trong một lớp 23 người thì đã có khoảng một nửa cơ hội có hai người trùng ngày sinh, dù một năm có 365 ngày. Nói cách khác, muốn an toàn với k món đồ thì số ngắn phải cỡ k^2 , chứ không phải cỡ k . Thứ hai, nếu có người biết quy tắc đánh số ngắn của bạn, họ có thể cố ý mang tới toàn những cái tên rơi vào cùng một ngắn, và tử của bạn thành một cái túi khổng lồ phải lục từng món. Cách chống rất đơn giản: mỗi lần mở tử buổi sáng, bạn trộn thêm một con số bí mật ngẫu nhiên vào quy tắc. Họ không đoán được nữa.

Một câu: Băm biến tên thành ngắn tử; mọi kỹ thuật còn lại chỉ để sống chung với chuyện hai tên rơi vào một ngắn.

Chỗ không rút gọn được: Không có hàm băm nào tránh được va chạm — số tên luôn nhiều hơn số ngắn; ta chỉ chọn được cách phân bố và cách phản ứng.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý tưởng cốt lõi của băm	Ép khoá bất kỳ thành chỉ số mảng bằng một hàm, để tra cứu thành truy cập trực tiếp. Va chạm là không tránh được.
••• Hai cách xử lý va chạm	Nói chuỗi (mỗi ô một danh sách, chịu tải > 1) và địa chỉ mở (dò sang ô kế tiếp, hợp cache, phải giữ tải $< 0,7$).

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao bảng băm bị đánh trượt	Hàm băm tắt định $\Rightarrow$ đối thủ dựng được tập khoá cùng ô $\Rightarrow$ mọi thao tác thành $O(n)$. Chứa bằng hạt giống ngẫu nhiên lúc chạy.
●●● Băm phổ dụng — ý chính	Ngẫu nhiên nằm ở <i>lựa chọn hàm băm của bạn</i> , không ở dữ liệu; nhờ đó đảm bảo vẫn đúng với dữ liệu thù địch.
●●● Nghịch lý ngày sinh	Va chạm đáng kể khi số phần tử đạt cỡ $\sqrt{m}$, không phải m . Xác suất có va chạm $\approx k^2/(2m)$.
●●● Hệ quả cho băm chuỗi	Modulo 10^9 chỉ an toàn tới cỡ $3 \cdot 10^4$ chuỗi; bài lớn cần hai modulo hoặc modulo cỡ 10^{18} , và cơ sở phải ngẫu nhiên.
●● Hệ số tải và mở rộng bảng	$\alpha = n/m$; vượt ngưỡng thì cấp phát bảng gấp đôi và băm lại, chi phí $O(n)$ nhưng khấu hao $O(1)$.
●● Bloom filter	Mảng bit + k hàm băm; “chắc chắn không có” hoặc “có lẽ có”. Dùng làm bộ lọc rẻ trước một phép tra cứu đắt.
●● Ba thứ băm không làm được	Không có thứ tự (không hỏi cận trên/dưới, không duyệt tăng dần); không đảm bảo trường hợp xấu nhất; $O(1)$ chỉ tính theo số phép băm, khoá dài vẫn tồn theo độ dài.
●● Khi nào băm là lựa chọn sai	Khi khoá là số nguyên trong khoảng nhỏ — mảng thường luôn thắng: nhanh hơn, đơn giản hơn, không va chạm.
●● Băm nhất quán	Xếp khoá và máy chủ lên vòng tròn, khoá thuộc về máy kế tiếp theo chiều kim đồng hồ; thêm máy chỉ làm $1/n$ số khoá đổi chỗ.
● Đếm phần tử phân biệt	Giá trị băm nhỏ nhất nhìn thấy được nói lên số phần tử phân biệt; HyperLogLog tinh chỉnh ý tưởng đó, vài KB cho hàng tỉ bản ghi.
● Băm mật mã khác băm thường	Yêu cầu không ai tạo được va chạm kể cả khi cố tình; chậm hơn nhiều nên không dùng cho bảng băm, nhưng cần cho định danh nội dung như trong git.
●● Băm cuckoo và băm bảng tra	Cuckoo: mỗi khoá chỉ nằm ở một trong hai ô nên tra cứu là $O(1)$ <i>trường hợp xấu nhất</i> , giá dồn sang phép chen và đòi tải dưới $1/2$. Bảng tra: hàm băm rẻ, chỉ đọc lập bậc ba, nhưng vẫn có chặn tập trung đã chứng minh.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên chọn kích thước bảng băm là số nguyên tố hay lũy thừa của 2?

| Lũy thừa của 2 cho phép thay phép chia lấy dư bằng phép AND bit, nhanh hơn đáng kể — nhưng nó chỉ giữ lại các bit thấp của mã băm, nên đòi hỏi hàm băm phải trộn tốt các bit. Số nguyên tố tha thứ hơn cho hàm băm kém vì phép lấy dư trộn mọi bit, nhưng phép chia chậm. Thực hành hiện đại nghiêng về lũy thừa của 2 kèm một bước trộn bit sau khi băm. Trong thi đấu, dùng lũy thừa của 2 kèm trộn hạt giống là lựa chọn an toàn.

2. Khi nào nên dùng map thay unordered_map trong C++?

| Bốn tình huống. Khi cần thứ tự hoặc truy vấn cận (lower_bound). Khi số phần tử nhỏ — dưới vài trăm thì hằng số của map không đáng kể và code an toàn hơn. Khi bài thi có nguy cơ bị dựng test phá và bạn không muốn viết hàm băm riêng. Và khi cần đảm bảo trường hợp xấu nhất. Ngược lại, với hàng triệu thao tác tra cứu thuần bằng khoá và có hàm băm tự viết, unordered_map hoặc một bảng địa chỉ mở tự cài sẽ nhanh hơn nhiều lần.

3. Băm chuỗi có thể thay hoàn toàn KMP và suffix array không?

| Thay được rất nhiều bài và thường ngắn hơn hẳn, nhưng có ba giới hạn thật. Một, nó là thuật toán xác suất — xác suất sai nhỏ nhưng khác không, và trong bài yêu cầu kết quả chắc chắn đúng thì đó là điều cần cân nhắc. Hai, một số bài cần cấu trúc bên trong của chuỗi (biên tuần hoàn, số chuỗi con phân biệt) mà băm không cho, phải dùng công cụ ở B/12. Ba, băm chuỗi so sánh được “bằng

nhau” nhưng không so sánh được “lớn hơn” theo thứ tự từ điển, trừ khi kết hợp thêm tìm kiếm nhị phân trên độ dài tiền tố chung.

4. Bloom filter chọn bao nhiêu hàm băm là tốt nhất?

| Có sự đánh đổi rõ: ít hàm băm thì mỗi phần tử bật ít bit nên mảng thưa hơn, nhưng mỗi truy vấn kiểm tra ít điều kiện nên dễ báo nhầm; nhiều hàm băm thì ngược lại. Với m bit và n phần tử, số hàm băm tối ưu tỉ lệ với m/n — cụ thể là $(m/n) \ln 2$, và tại điểm tối ưu thì khoảng một nửa số bit được bật. Điều đáng nhớ về mặt trực giác: mảng đầy quá nửa thì cấu trúc bắt đầu mất tác dụng.

5. Vì sao không dùng luôn hàm băm mật mã cho bảng băm để khỏi lo bị tấn công?

| Vì nó chậm hơn nhiều bậc — được thiết kế để chống lại đối thủ có nhiều tài nguyên, nên có tính tốn nhiều phép toán. Với bảng băm, thứ bạn cần chỉ là đối thủ *không đoán trước được*, mà điều đó đạt được rẻ hơn nhiều bằng một hàm băm nhanh cộng một hạt giống ngẫu nhiên bí mật. Dùng đúng công cụ cho đúng mức đe dọa là nguyên tắc chung, và dùng quá mức cũng là một dạng sai.

6. Trong hệ thống thật, lỗi liên quan tới băm hay gặp là gì?

| Ba lỗi phổ biến. Một, dùng mã băm làm định danh lâu dài rồi gặp va chạm khi dữ liệu lớn lên — đúng bấy ngày sinh, và nó xuất hiện muộn nên rất khó truy. Hai, giả định thứ tự duyệt của cấu trúc băm là ổn định giữa các lần chạy; nhiều thư viện hiện đại cố tình xáo thứ tự đó, và code phụ thuộc vào nó sẽ hỏng không đều. Ba, dùng hash mod n cho phân mảnh rồi phải mở rộng cụm máy — lúc đó gần như toàn bộ dữ liệu phải di chuyển, và cách sửa là băm nhất quán.

7. Có cấu trúc nào cho $O(1)$ trường hợp xấu nhất chứ không chỉ trung bình không?

| Có: băm hoàn hảo. Với tập khoá *tĩnh* biết trước, người ta xây được một hàm băm không có va chạm nào, cho tra cứu $O(1)$ đảm bảo; kỹ thuật hai tầng cổ điển đạt được điều đó với bộ nhớ tuyến tính. Hạn chế là tập khoá phải cố định, nên nó dùng cho từ khoá ngôn ngữ lập trình, bảng tra cố định trong hệ thống nhúng — những nơi dữ liệu không đổi sau khi biên dịch.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Đếm số phần tử phân biệt trong mảng	CSES “Distinct Numbers”[4]	So sánh ba cách: sắp xếp, tập hợp có thứ tự, tập hợp băm — và đo thời gian thật.
● Hai số có tổng bằng x , mảng chưa sắp	LeetCode “Two Sum”	Bảng băm một lượt; đối chiếu với cách hai con trỏ ở $B/06$ và nêu điều kiện chọn mỗi cách.
● Nhóm các chuỗi đảo chữ với nhau	LeetCode “Group Anagrams”	Thiết kế khoá băm là bước quyết định — chuỗi đã sắp hay vector đếm 26 phần tử.
●● Đếm số đoạn con có tổng bằng x	CSES “Subarray Sums II”[4]	Tổng tiền tố cộng bảng băm; chú ý tổng có thể âm và phải dùng kiểu 64 bit.
●● Đoạn con dài nhất có tổng bằng 0	Bài tập kinh điển	Cùng mẫu: lưu lần xuất hiện <i>đầu tiên</i> của mỗi giá trị tổng tiền tố.
●● Cài một bảng băm địa chỉ mở của riêng bạn	Bài tập cài đặt	So thời gian với thư viện chuẩn trên 10^7 thao tác; đây là bài tập cho thấy rõ nhất tác động của cache.
●● Dựng bộ test làm chậm unordered_map	Bài tập an ninh	Chọn khoá cùng số dư theo kích thước bảng; sau đó chứa bằng hàm băm có hạt giống và đo lại.
●●● So sánh mọi cặp đoạn con bằng băm chuỗi	CSES nhóm bài chuỗi[4]	Hai modulo; tính xác suất va chạm cho k chuỗi trước khi cài, đừng đoán.
●●● Tìm chuỗi con lặp lại dài nhất bằng băm và nhị phân trên độ dài	Bài tập kinh điển	Kết hợp băm với nhị phân trên đáp án ($C/19$); chú ý tính đơn điệu của điều kiện.
●●● Cài Bloom filter và đo tỉ lệ báo nhầm theo số hàm băm	Bài tập thực nghiệm	Vẽ đường cong tỉ lệ báo nhầm theo k ; so với dự đoán lý thuyết và giải thích chỗ lệch.
●●● Cài băm nhất quán với nút ảo, mô phỏng thêm/bớt máy chủ	Bài tập hệ thống	Đo tỉ lệ khoá phải di chuyển; so với mod n để thấy khác biệt bằng số liệu.

Kết chương và đọc tiếp

Băm đối ba thứ — thứ tự, đảm bảo xấu nhất, và tính không đoán trước — để lấy $O(1)$ trung bình. Hai điều đáng mang theo: nghịch lý ngày sinh nói ngưỡng an toàn là $\sqrt{m}$ chứ không phải m ; và mọi phân tích “trung bình” đều sụp nếu dữ liệu do đối thủ chọn, nên hạt giống ngẫu nhiên không phải tùy chọn mà là một phần của lời giải.

Chương tiếp theo trả lời câu hỏi ngược lại: nếu bạn cần thứ tự thì phải trả thêm bao nhiêu, và các cấu trúc cây tìm kiếm giữ cân bằng bằng cách nào (B/09). Bạn sẽ thấy một mẫu quen thuộc lặp lại: các cấu trúc thắng cuộc trong thực tế thường là các cấu trúc *ngẫu nhiên hoá*, vì lý do giống hết lý do trong chương này.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Pagh & Rodler, “Cuckoo Hashing”	Journal of Algorithms, 2004	Bản viết rất gọn của một ý tưởng đẹp. Đọc phần mô tả thuật toán chèn và điều kiện hệ số tải; phần chứng minh đọc sau.
nền tảng	Pătraşcu & Thorup, “The Power of Simple Tabulation Hashing”	Journal of the ACM, 2012	Đọc phần mở đầu và phát biểu các định lý — đủ để hiểu vì sao một hàm băm chỉ đọc lập bậc ba lại dùng được ở nơi lý thuyết cũ đòi bậc cao.
tra khi cần	Bloom, “Space/Time Trade-offs in Hash Coding with Allowable Errors”	Communications of the ACM, 1970	Bốn trang, bài gốc của Bloom filter; đọc để thấy động cơ ban đầu là tiết kiệm bộ nhớ cho từ điển chính tả.
tra khi cần	Flajolet, Fusy, Gandouet, Meunier, “HyperLogLog”	Conference on Analysis of Algorithms, 2007	Đọc khi cần đếm phần tử phân biệt ở quy mô lớn; phần phân tích là ví dụ mẫu mực về xác suất trong thiết kế cấu trúc.

Nguồn

- [1] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [2] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [3] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [4] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).